

LUIS REIS 22-02-2023

SUMMARIZER



java.net.InetAddress

- Represents an IP Address
- Performing DNS resolutions
- Storing results in a local cache
- Testing for host reachability
- This class provides methods to get the IP of any hostname, for example *www.wikipedia.com*

Networking classes

- Socket
- ServerSocket
- URL
- DatagramSocket
- DatagramPacket

java.net.InetAddress



```
public static void main(String[] args) {  
  
    String host = getHost();  
    try {  
        InetAddress address = InetAddress.getByName(host);  
        System.out.println("Testing reachability for " + address.getHostAddress() + address);  
        System.out.println(address.isReachable(3000));  
  
    } catch (IOException e) {  
        System.out.println("I fuck*d up!");  
    }  
  
}
```

java.net.InetAddress

```
private static String getHost() {  
    Scanner read = new Scanner(System.in);  
    System.out.println("HOST? ");  
    return read.nextLine();  
}
```

HOST?

www.wikipedia.com

Testing reachability for 185.15.58.226www.wikipedia.com/185.15.58.226

true

Sockets

It is a communication mechanism, normally used to run a model, that allows the exchange of “messages” between a server and a client.



Sockets

DatagramSockets - UDP

Sending and receiving UDP datagrams to/from a specified host and port.

Socket - TCP

Capable of connecting to a specified server and port.

Server Socket - TCP

Used to wait for requests to come in over the network.

Sockets

TCP



- Slower but more reliable transfers
- Typical Applications:
 - File Transfer Protocol (FTP)
 - Web Browsing
 - Email



unicast

UDP



- Faster but not guaranteed transfers ("best effort")
- Typical Applications:
 - Live Streaming
 - Online Games
 - VoIP



unicast



multicast



broadcast

UDP - Socket - Client



```
String host = getHost();
byte[] receiveR = new byte[1024];
byte[] sendR = new byte[2048];
int portNumb = getPort();
String getMessage = enterMessage();

sendR = getMessage.getBytes();
```



```
private static String getHost() {
    Scanner read = new Scanner(System.in);
    System.out.println("HOST? ");
    return read.nextLine();
}

private static int getPort() {
    Scanner read = new Scanner(System.in);
    System.out.println("Port? ");
    return read.nextInt();
}

private static String enterMessage() {
    Scanner read = new Scanner(System.in);
    System.out.println("Enter message: ");
    return read.nextLine();
}
```

UDP - Socket - Client

```
try {
    DatagramSocket socket = new DatagramSocket();
    InetAddress.getAllByName(host);
    DatagramPacket sendPacket = new DatagramPacket(sendR, sendR.length,
    InetAddress.getByNme(host), portNumb);
    socket.send(sendPacket);

    DatagramPacket receivedPacket = new DatagramPacket(recvieR, recvieR.length);
    socket.receive(receivedPacket);

    String msgReceived = new String (receivedPacket.getData());
    System.out.println(msgReceived);
} catch (IOException e) {
    System.out.println("Wrong door my dear");
}
```

UDP - Socket - Server



```
byte[] sendR = new byte[1024];  
int portNumber = 20;  
byte[] recevR = new byte[1024];  
String hostName = "localhost";
```



```
try {  
    DatagramSocket socket = new DatagramSocket(20);  
  
    DatagramPacket receivedPacket = new DatagramPacket(recevR,  
recevR.length);  
    socket.receive(receivedPacket);  
  
    String msgReceived = new String  
receivedPacket.getData().toUpperCase();  
    System.out.println(msgReceived);  
  
    sendR = msgReceived.getBytes();  
  
    DatagramPacket sendPacket1 = new DatagramPacket(sendR, sendR.length,  
receivedPacket.getAddress(), receivedPacket.getPort());  
    socket.send(sendPacket1);  
  
}
```

UDP - Socket

Client

HOST?

127.0.0.1

Port?

20

Enter message:

Tomorrow the summarizer is yours!

Server

TOMORROW THE SUMMARIZER IS YOURS!



TCP - Socket - Client

```
String hostName = "localhost";

String word = "lisbon" ;

try {
    Socket clientSocket = new Socket(hostName, 20);
    BufferedReader in = new BufferedReader(new
InputStreamReader(clientSocket.getInputStream()));

    PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);

    out.println(word);
}
```

TCP - Socket - Server

```
int portNumber = 20;

try {
    ServerSocket serverSocket = new ServerSocket(portNumber);
    Socket clientSocket = serverSocket.accept();

    PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
    BufferedReader in = new BufferedReader(new
    InputStreamReader(clientSocket.getInputStream()));

    String line = in.readLine();

    while(line != null){
        System.out.println(line);
        line = in.readLine();
    }
}
```




THANK YOU

FOR SUPPORTING ME